FinTrackPro

A Java Full-Stack Personal Expense Tracking and Budget Management System

Project by

Submitted by
Kodadhala Vikas
kodadhalavikas@gmail.com
August 2026

Table of Contents

1. Introduction

Managing personal finances is a challenge that affects nearly everyone, regardless of income level or profession. Without a clear, organized way to record income and expenses, it becomes difficult to understand spending habits, plan for future expenses, or build savings consistently. FinTrackPro was developed to address this everyday problem through a simple, secure, and visually informative web application.

This project began as an analysis and extension exercise: an existing open-source Spring Boot expense tracker was studied, run locally, and then progressively rebuilt with production-relevant features — authentication and password security, data visualization, budget management, savings goals, recurring expenses, data export, and a personalized user interface — resulting in FinTrackPro, a considerably more complete and secure application than the original starting point.

2. Problem Statement

Most individuals track their day-to-day expenses inconsistently — using paper notes, generic notes apps, or not at all. This leads to a lack of visibility into spending patterns, an inability to set or respect a monthly budget, and difficulty in saving toward specific financial goals. There is a need for a lightweight, easy-to-use application that centralizes this information, presents it visually, and actively alerts the user when their spending approaches or exceeds a set limit.

3. Objective of Project

- To design and develop a secure, full-stack web application for tracking personal income and expenses.
- To implement user authentication using industry-standard practices (password hashing and JWT-based sessions).
- To provide meaningful visual insight into spending through charts and periodic summaries.
- To help users set, monitor, and stay within a monthly budget.
- To support long-term financial planning through savings goals and recurring expense tracking.
- To allow users to export their financial data in commonly used formats (CSV, Excel, PDF).

4. Goal of Project

The overarching goal of FinTrackPro is to give an individual user a single, self-hosted place to record every transaction, understand where their money goes each month, and make informed decisions about saving and spending — all through an interface that is fast, secure, and pleasant to use on a day-to-day basis.

5. Problem Identification

During the initial analysis of the base project, several concrete gaps were identified that directly motivated FinTrackPro's feature set:

- No password encryption — credentials were stored and compared as plain text.
- No session or token-based authentication — any client could call another user's data endpoints directly.
- No categorization of transactions, making spending analysis impossible.
- No dashboard-level insight — only a running total was shown, with no charts or time-based summaries.
- No budgeting, alerts, or savings tracking of any kind.
- No way to search, filter, or export transaction history.
- No personalization — no currency preference, theme, or profile information.

6. Existing System

The starting point was a minimal Java, Spring Boot and MySQL application exposing basic REST endpoints for user registration, login, and transaction CRUD operations, paired with a static HTML/CSS/JavaScript frontend. While functionally able to record and total transactions, the existing system had no meaningful security model, no data visualization, and none of the budgeting or personalization features expected of a modern finance tool. Its scope was limited to being a proof-of-concept CRUD demonstration rather than a usable application.

7. Proposed System

FinTrackPro was proposed and implemented as a layered enhancement of the existing system, adding functionality in incremental, independently verified phases:

- Phase 1 & 2 — Security overhaul (BCrypt password hashing, JWT authentication, per-user data isolation) and transaction categorization.

- Phase 3 — Dashboard summaries (weekly/monthly/yearly) and three interactive charts (pie, bar, line) built with Chart.js.
- Phase 4 — Search, category filtering, date-range filtering, and multi-field sorting of transaction history.
- Phase 5 — Monthly budget configuration with live remaining-budget calculation and threshold-based alerts.
- Phase 6 — User profile personalization: custom profile photo, currency preference, and a dark mode theme.
- Phase 7 — Client-side export of transaction data to CSV, Excel, and PDF formats.
- Phase 8 — Recurring transaction support and savings goals with progress tracking.

Each phase was implemented, deployed to a local environment, and functionally verified end-to-end (registration through to database persistence) before the next phase began, ensuring a stable and traceable development process.

8. Requirements

8.1 Software Requirements

Component	Technology / Version
Programming Language	Java 17
Backend Framework	Spring Boot 3.5.x
Persistence	Spring Data JPA (Hibernate)
Security	Spring Security, JJWT (JSON Web Tokens)
Database	MySQL 8.0
Build Tool	Apache Maven 3.9+
Frontend	HTML5, CSS3, JavaScript (ES6)
Charting Library	Chart.js 4.x
Export Libraries	SheetJS (xlsx), jsPDF, jsPDF-AutoTable
IDE	Visual Studio Code
Version Control	Git & GitHub
Operating System (dev)	Windows 10 / 11

8.2 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 2.0 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	500 MB free disk space
Network	Internet connection (for CDN libraries and package downloads)
Display	1366 x 768 resolution or higher

9. Design and Implementation

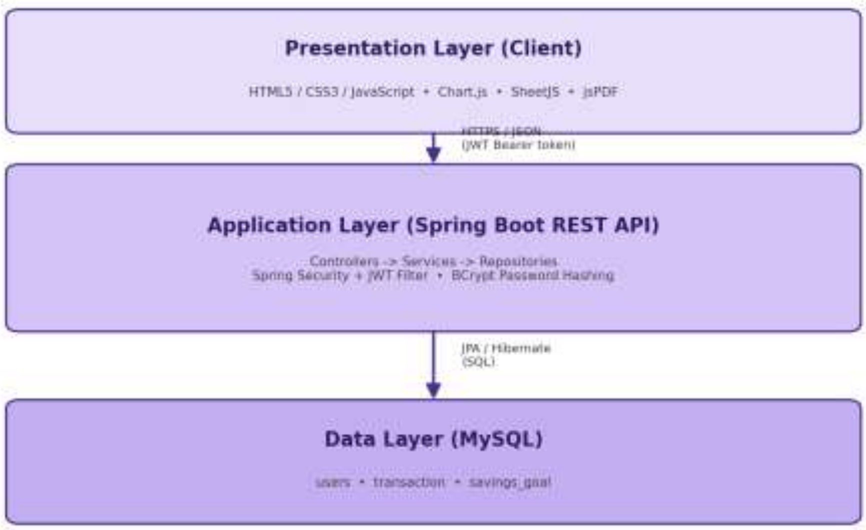
9.1 Design

FinTrackPro follows a classic client-server design. The client is a static, framework-free web frontend responsible entirely for presentation and user interaction; the server is a stateless REST API responsible for business logic, security enforcement, and persistence. All communication between the two happens over JSON via HTTP, with every protected request authenticated using a JSON Web Token issued at login.

Within the backend, responsibilities are separated using the standard Controller-Service-Repository pattern: controllers handle HTTP concerns and ownership checks, services contain business logic, and repositories (Spring Data JPA interfaces) handle database access, keeping each layer independently testable and replaceable.

9.2 Architecture

The diagram below illustrates the three-tier architecture used throughout the application.



9.3 Implementation

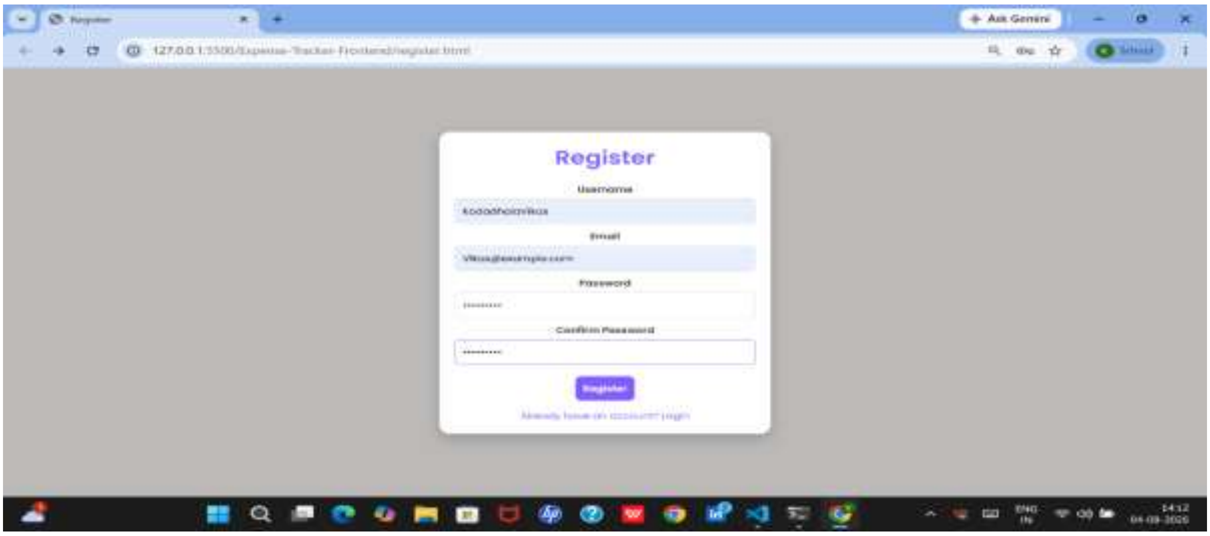
The backend exposes the following key REST endpoints, all secured by a custom JWT authentication filter placed ahead of Spring Security's authorization chain:

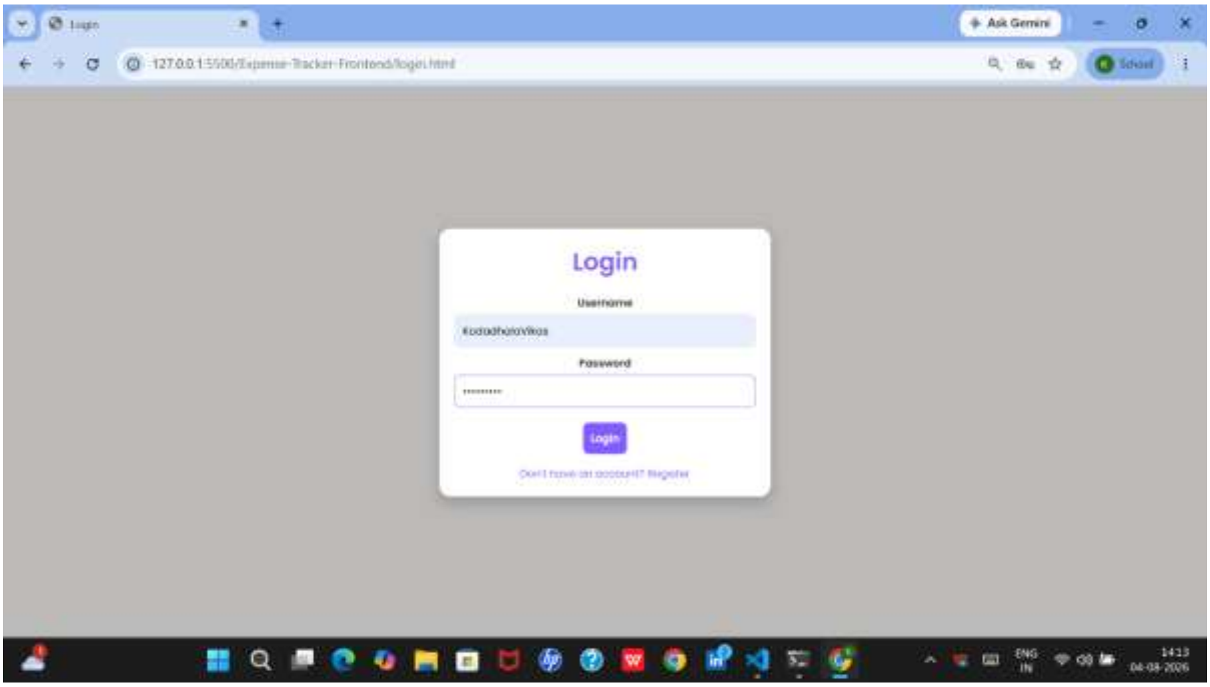
Endpoint	Purpose
POST /ExpTrack/register	Register a new user (password hashed with BCrypt before storage)
POST /ExpTrack/login	Authenticate and issue a signed JWT
GET / POST / DELETE /ExpTrack/transactions/{username}	List, add, or delete transactions, with category, recurrence, and date

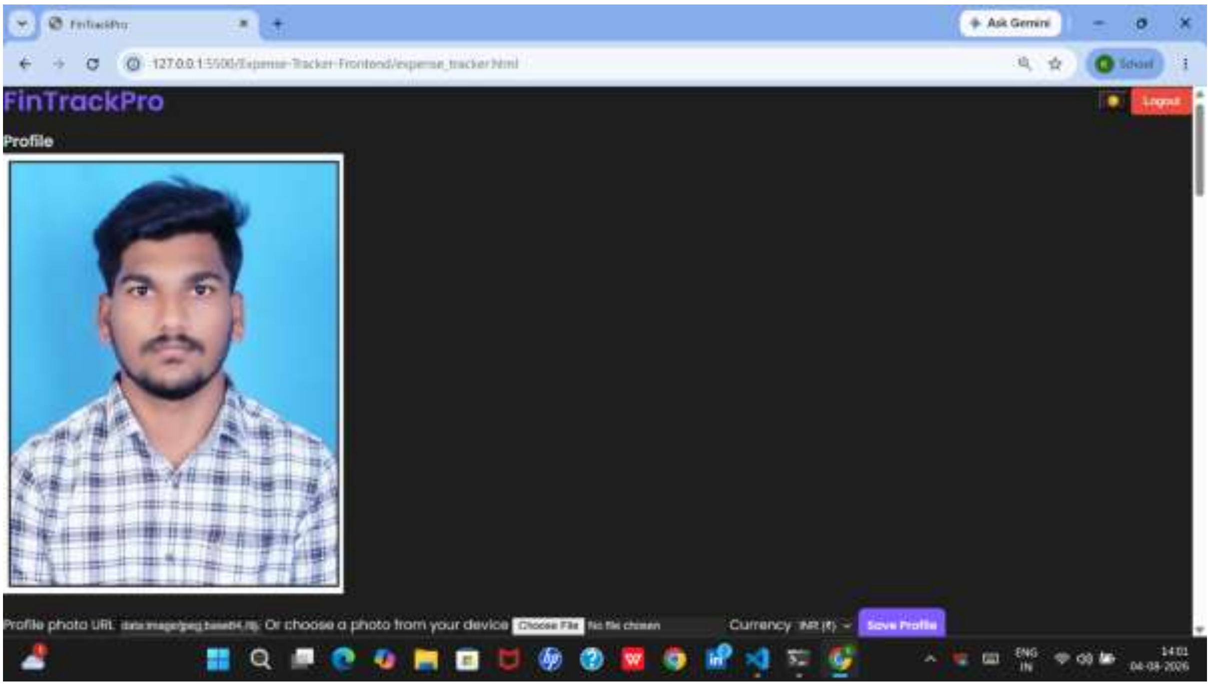
GET / PUT /ExpTrack/budget/{username}	Retrieve or update the monthly budget
GET / PUT /ExpTrack/profile/{username}	Retrieve or update avatar and currency preference
GET / POST /ExpTrack/goals/{username}	List or create savings goals
PUT / DELETE /ExpTrack/goals/{username}/{id}	Contribute to or delete a savings goal

On the frontend, all dashboard computations — weekly/monthly/yearly summaries, chart data aggregation, search/filter/sort, and CSV/Excel/PDF export — are performed client-side against the transaction list already fetched from the API, minimizing server load and keeping the interface responsive.

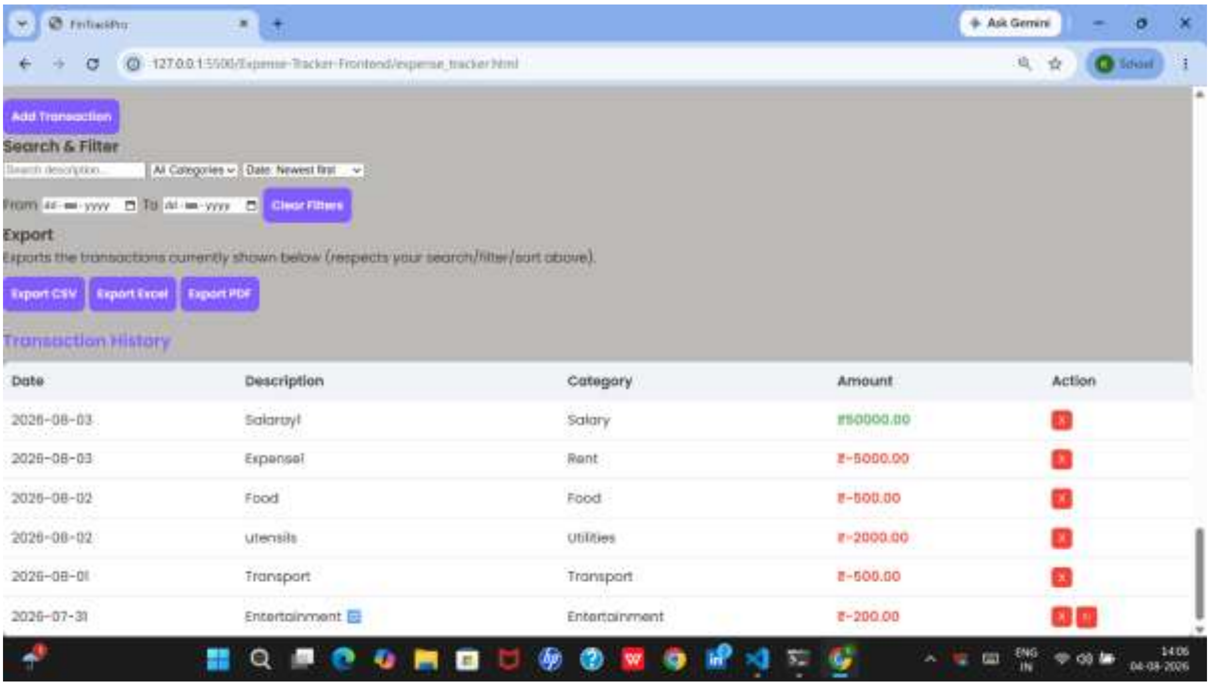
9.4 Screenshots of Application

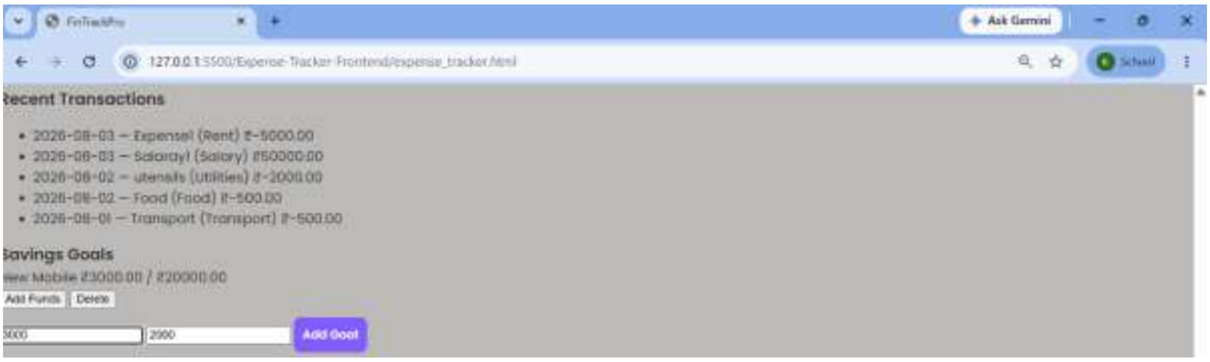




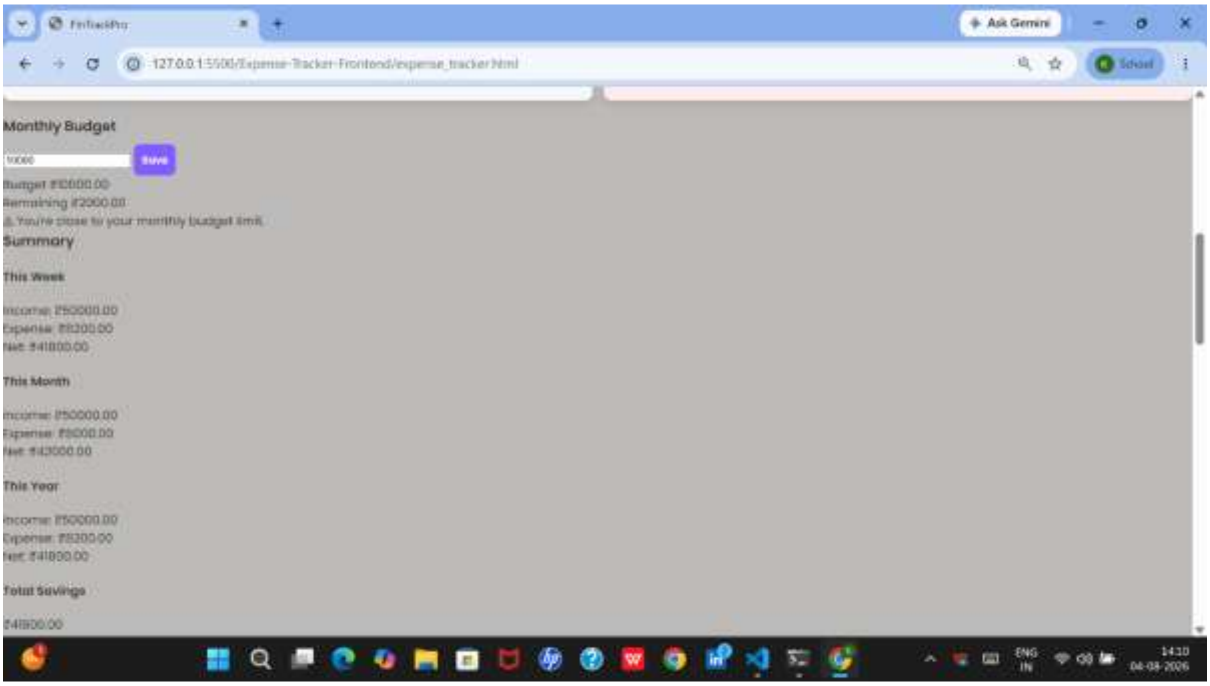

FinTrackPro
127.0.0.1:5500/Expense-Tracker-Frontend/expense_tracker.html
Ask Gemini
Logout
FinTrackPro
Profile
Profile photo URL
Or choose a photo from your device
Choose File
No file chosen
Currency INR (₹)
Save Profile

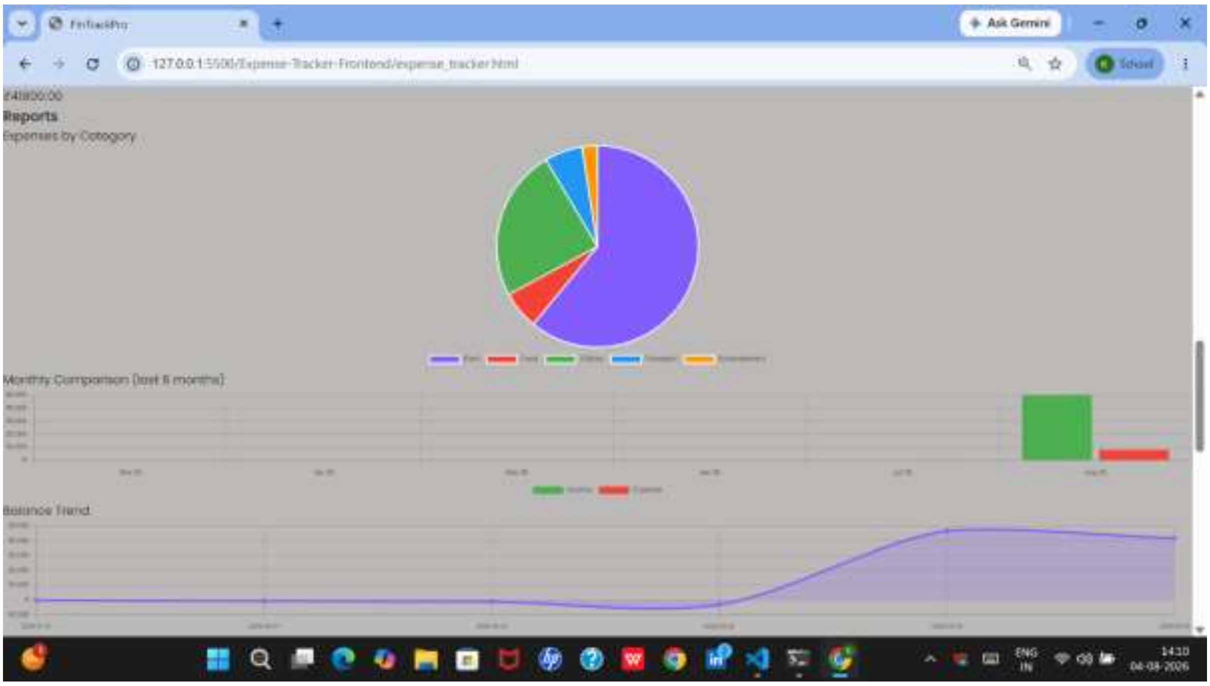

FinTrackPro
127.0.0.1:5500/Expense-Tracker-Frontend/expense_tracker.html
Ask Gemini
Add Transaction
Description
Festival Shopping
Amount (negative = expense, positive = income)
2000
Category
Shopping
Date
04-08-2026
Repeat this transaction
Add Transaction

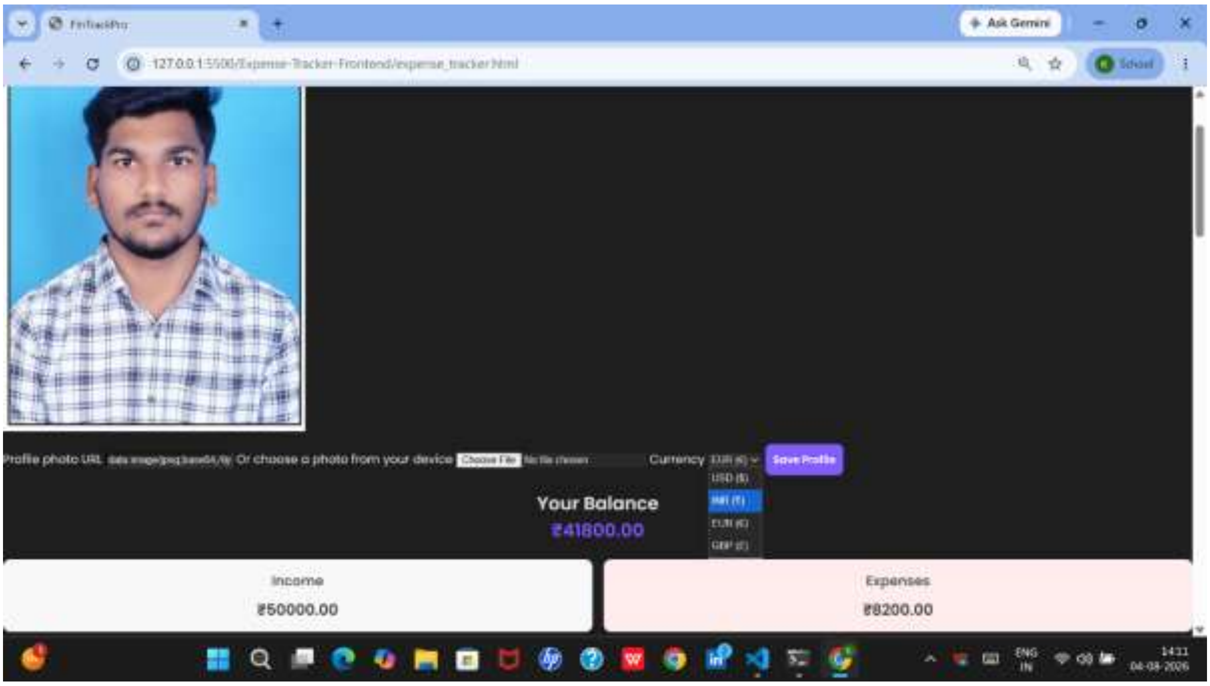

FinTrackPro
127.0.0.1:5500/Expense-Tracker-Frontend/expense_tracker.html
Ask Gemini
Add Transaction
Search & Filter
All Categories
Date: Newest first
From dd-mm-yyyy To dd-mm-yyyy
Clear Filters
Export
Exports the transactions currently shown below (respects your search/filter/sort above).
Export CSV
Export Excel
Export PDF
Transaction History
Date Description Category Amount Action
2026-08-03 Salary Salary ₹50000.00
2026-08-03 Expense Rent ₹-5000.00
2026-08-02 Food Food ₹-500.00
2026-08-02 utensils Utilities ₹-2000.00
2026-08-01 Transport Transport ₹-500.00
2026-07-31 Entertainment Entertainment ₹-200.00

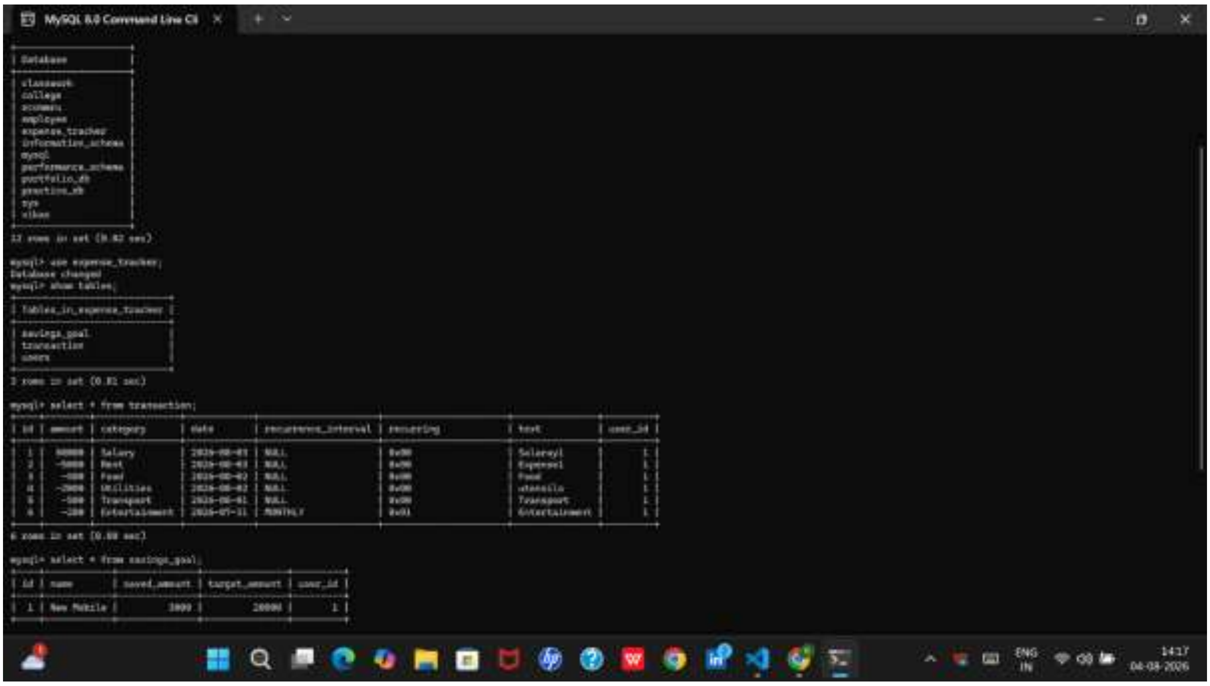

Recent Transactions
2026-08-03 — Expense1 (Rent) ₹-5000.00
2026-08-03 — Salary1 (Salary) ₹50000.00
2026-08-02 — utensils (Utilities) ₹-2000.00
2026-08-02 — Food (Food) ₹-500.00
2026-08-01 — Transport (Transport) ₹-500.00
Savings Goals
Add Funds
Delete
Add Goal


Search & Filter
Clear Filters
Export
Export CSV
Export Excel
Export PDF
Transaction History
Date
Description
Category
Amount
Action
2026-08-02
Food
Food
₹-500.00


Monthly Budget
Save
Summary
This Week
This Month
This Year
Total Savings


FinTrackPro
Ask Gemini
127.0.0.1:5500/Expense-Tracker-Frontend/expense_tracker.html
Reports
Expenses by Category
Monthly Comparison (last 6 months)
Balance Trend


FinTrackPro
Ask Gemini
127.0.0.1:5500/Expense-Tracker-Frontend/expense_tracker.html
Profile photo URL
Or choose a photo from your device
Currency
Save Profile
Your Balance
₹41800.00
Income
₹50000.00
Expenses
₹8200.00



10. Results

The completed FinTrackPro application was tested end-to-end across all implemented phases, with each feature verified against a live MySQL database rather than mocked data. Verified outcomes include:

- Successful registration and login with BCrypt-hashed passwords and JWT issuance.
- Enforcement of per-user data access — a user cannot view or modify another user's transactions, budget, or goals.
- Accurate real-time calculation of balance, income, expense, and period-based (weekly/monthly/yearly) summaries.
- Correct rendering of pie, bar, and line charts reflecting live transaction data.
- Functional search, category filtering, date-range filtering, and sorting of transaction history.
- Working budget configuration with correctly triggered 'approaching' and 'exceeded' alert states.
- Successful data export to CSV, Excel, and PDF formats, respecting active filters.
- Functional recurring transaction generation and savings goal progress tracking.
- Personalization features — custom profile photo, currency switching, and dark mode — functioning correctly across the dashboard.